

Global Fabric Day 2026 – Madrid

Viernes, 26 de Junio, 2026



Stop Clicking

**Domina Microsoft Fabric con
Fabric as Code, Copilot y GSD**

David Oliva · Jorge Fernández



Global Fabric Day 2026 – Madrid

Agradecimiento especial a



Microsoft



bravent



TD SYNnex

¡Atent@s hasta el final!



Global Fabric Day 2026 – Madrid



David Oliva

Cloud Solution
Architect



Jorge Fernández

Technical Lead



Agenda

Del problema a una demo reproducible

- 1 Por qué Fabric as Code
- 2 Dónde encaja GSD
- 3 Cómo modelamos artefactos Fabric
- 4 Demo end-to-end
- 5 Gobierno, límites y próximos pasos



Seguimos haciendo “click-ops” en datos

laC es estándar... pero muchos entornos Fabric aún se operan manualmente

- 01 Cambios difíciles de revisar
- 02 Promoción entre entornos frágil
- 03 Poca trazabilidad de decisiones
- 04 IA sin contexto produce deuda



Objetivo de la sesión

Convertir una solución de datos en un flujo declarativo, versionado, verificable y desplegable.

Modelo mental

GSD orquesta la intención; Fabric materializa los artefactos



GSD

Discusión guiada
Plan de fases
Verificación continua
Contexto persistente

Microsoft Fabric

Lakehouse
Notebook
Pipeline
Semantic Model

DevOps

Git
Pull Request
Validación
Promoción Dev/Test/Prod

Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core



GSD aplicado a Fabric

Un bucle disciplinado para que la IA no improvise arquitectura



STATE.md mantiene decisiones, alcance y supuestos.
Las fases obligan a separar intención, diseño, ejecución y prueba.
Copilot trabaja contra contexto explícito, no contra prompts sueltos.

ejemplo de orden de trabajo
/gsd-discuss solución Fabric ventas medallion
/gsd-plan item definitions + CI/CD
/gsd-execute crear artefactos y tests
/gsd-verify validar estructura y despliegue

Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core

Arquitectura de trabajo

Una cadena reproducible de extremo a extremo



El repositorio se convierte en la frontera de gobierno: todo cambio importante pasa por spec, revisión y despliegue controlado.

Repositorio de la demo

Estructura simple, legible y preparada para automatización

```
fabric-gsd-demo/  
├── .gsd/  
│   ├── STATE.md  
│   └── phases/  
├── src/fabric/  
│   ├── SalesLakehouse.Lakehouse/  
│   ├── IngestSales.DataPipeline/  
│   ├── TransformSales.Notebook/  
│   └── SalesModel.SemanticModel/  
├── infra/  
│   ├── workspace.parameters.json  
│   └── deployment.yml  
└── tests/  
    └── validate-definitions.ps1
```

Nombres de ítems coherentes con Fabric Git Integration.

Parámetros por entorno separados de la definición.

Validaciones previas al despliegue para evitar cambios invisibles.



Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core

Demo: solución de ventas medallion

Un caso pequeño, pero suficientemente real para mostrar el flujo completo



Resultado: una versión inicial desplegable de Lakehouse + Pipeline + Notebook + Modelo semántico.

Demo 1 · De requisito a especificación

GSD convierte una idea de negocio en trabajo ejecutable

Actúa como arquitecto Fabric.
Necesitamos una solución de ventas con:

- arquitectura medallion
- ingestión incremental
- notebooks versionados
- modelo semántico para reporting
- despliegue Dev → Test

Decisiones explícitas antes de crear artefactos.

Supuestos y riesgos registrados.

Backlog técnico por fase, no una lista caótica de prompts.

Salida esperada: STATE.md + PLAN.md + criterios de verificación

Demo 2 · Generar Fabric Item Definitions

La IA acelera; la definición versionada manda

```
SalesLakehouse.Lakehouse/  
.platform  
lakehouse.metadata.json
```

```
TransformSales.Notebook/  
.platform  
notebook-content.py
```

```
IngestSales.DataPipeline/  
.platform  
pipeline-content.json
```

Qué enseñamos en pantalla

Copilot crea estructura inicial.

GSD verifica consistencia con la spec.

Los artefactos se revisan en Git, no solo en el portal.

El portal deja de ser la fuente de verdad.

Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core

Demo 3 · Pull Request como control de calidad

Cada cambio deja rastro técnico y funcional



```
git checkout -b feature/sales-lakehouse  
git add .  
git commit -m "Add sales Fabric definitions"  
git push -u origin feature/sales-lakehouse
```

Checklist de naming, parámetros y dependencias.
Diffs de definiciones revisables por el equipo.
Aprobaciones antes de promover a Test/Prod.

Demo 4 · Despliegue automatizado

Promoción controlada sin abrir diez pantallas del portal

steps:

- validate definitions
- replace environment parameters
- authenticate with service principal
- deploy item definitions
- smoke test workspace

Dev

Test

Prod

**Opciones: Fabric REST APIs, Bulk Import
Item Definition API, fabric-cicd o pipelines
de despliegue según madurez del cliente.**

Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core

Copilot: prompts que sí ayudan

Menos “hazme algo”, más contexto, contrato y verificación

Contexto:

Lee .gsd/STATE.md y src/fabric/.

Tarea:

Crea el pipeline IngestSales con parámetros por entorno.

Restricciones:

No hardcodees workspace IDs.

Mantén nombres Fabric compatibles con Git.

Verifica:

Devuelve archivos modificados y riesgos.

Anclar siempre al estado GSD.

Pedir salida verificable.

Separar generación de validación.

Hacer explícitas las reglas de entorno y naming.



Gobierno y seguridad

El enfoque declarativo solo funciona si el proceso está protegido

- A** Identidad: service principal, permisos mínimos y secretos fuera del repo
- B** Entornos: parámetros, naming y sustitución de IDs controlada
- C** Calidad: validaciones antes del deploy y smoke tests después
- D** Gobierno: revisión por PR y trazabilidad requisito
→ spec → release



Dónde hay que ser prudentes

La demo enseña el patrón; en producción hay matices

La madurez de Git/API puede variar por tipo de ítem Fabric. Algunos artefactos requieren manejo fino de dependencias e IDs entre workspaces.

La IA debe generar borradores, no saltarse revisión ni seguridad. Preparar un “plan B” en demo: repo ya generado + deploy prevalidado.

Mensaje clave

**No vendemos magia.
Vendemos un flujo repetible,
auditable y escalable.**



Referencias: Microsoft Learn Fabric REST APIs / Git integration · github.com/open-gsd/gsd-core

Qué te llevas

Tres ideas para empezar el lunes

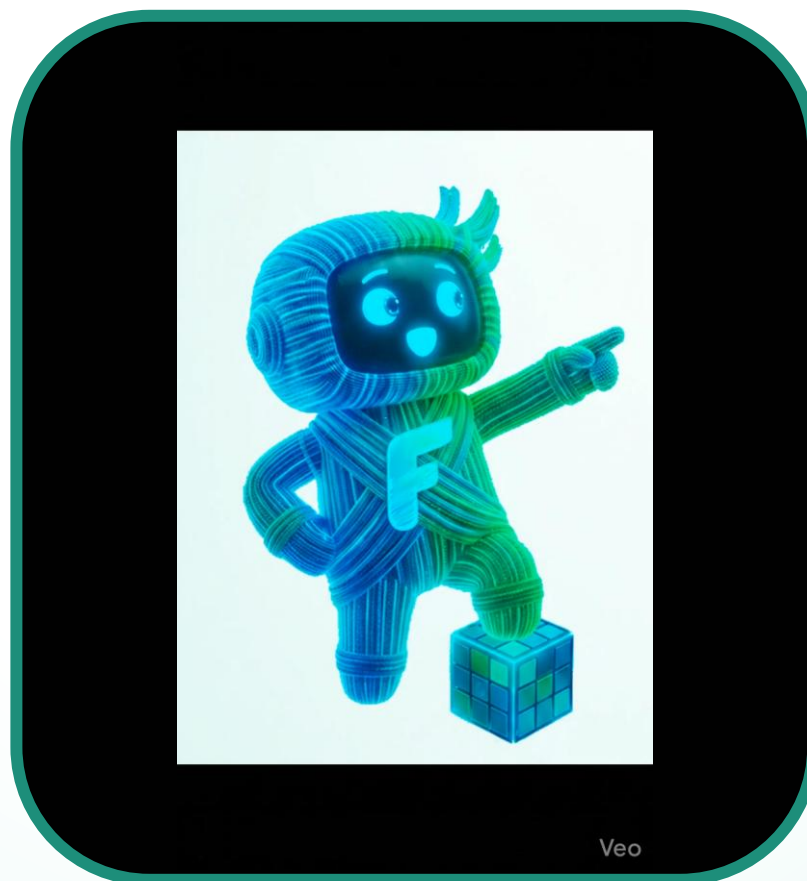
- 1** El portal no debe ser la fuente de verdad.
- 2** GSD ordena el contexto para que Copilot trabaje con intención.
- 3** Fabric as Code es DevOps aplicado a plataformas de datos.



Global Fabric Day 2026 – Madrid



<Demo>



¿Preguntas?

Gracias · Global Fabric Day 2026 – Madrid

Repositorio demo: github.com/davidop/fabric-gsd-demo

GSD Core: github.com/open-gsd/gsd-core



Únete a la comunidad



Global Fabric Day 2026 – Madrid

A continuación:

- **Track 1:**
 - Planning in Fabric IQ: del Excel al futuro
 - Mónica Mesa / Oscar Garzón
- **Track 2:**
 - Del ETL al AI-ETL: transforma tus datos con Fabric AI Functions
 - Toni Ferrá / María Soto

